RELATÓRIO DE CHECK POINT

Projeto Sistemas Embarcados com ESP32 | Validação da evolução do projeto

Disciplina	Sistemas Embarcados – CESAR School
Professores	Bella Nunes | Jymmy Barreto
Data do Check Point	17/11/2026
Grupo (nº)	Grupo 7
Título do projeto	DENGUERADAR
Resumo do projeto	Sistema IoT que fotografa ovitrampas com uma ESP32-CAM, as fotos chegam ao PC via MQTT a um serviço python que conta os ovos com YOLO, e um dashboard web dispara as capturas e mostra os resultados em tempo real.
Representante do grupo	Igor Andrade Canuto Nardelli Couto

Integrantes

Nº	Nome completo	Usuário no GitHub	Responsabilidade no projeto
1	Arthur Leal Pacheco	arthurlealp	Dashboard e Docker
2	Bruno Carvalho Feitosa	BrunofCarvalhoo	Dashboard e planejamento do funcionamento do modelo
3	Guilherme Coutinho Almeida	GuiCalmeida87	Treino do modelo e e-mail
4	Igor Andrade Canuto Nardelli Couto	IgorACNC	Treino do modelo e montagem física da câmera
5	Lucas Souto Maior Caldas	Souto1002	Dashboard
6	Raul Vitor Ferraz Silva	raulferraz85	Dashboard
7	William Moreira Jaruzo De Souza	william-mjsouza	Análise de desempenho do modelo e cuidar da comunicação MQTT

Como preencher

Substitua os textos cinza entre colchetes [ ], marque as opções com ☒ e insira imagens nas áreas pontilhadas. Seja objetivo: o foco é mostrar o quanto o projeto evoluiu até aqui. O painel da última página é de uso dos docentes.

1. Prototipagem

1.1 Uso dos sensores/atuadores

Orientação: *Mantenha apenas os componentes que o grupo já está usando e acrescente os que não constam na lista.*

Componente	Tipo	Função no projeto	Pino(s) / GPIO	Situação
ESP32-CAM AI Thinker	Controlador	Wi-fi, MQTT, task FreeRTOS de captura		■ OK ☐ Parcial ☐ Pendente
Câmera OV2640	Sensor	Fotografa a palheta e envia via MQTT	SCCB: SDA 26 / SCL 27; XCLK 0; PWDN 32; D0–D7: 5, 18, 19, 21, 36, 39, 34, 35; VSYNC 25; HREF 23; PCLK 22	■ OK ☐ Parcial ☐ Pendente
[Outro componente]	*[Controlador, Sensor,Atuador]*			☐ OK ☐ Parcial ☐ Pendente
[Outro componente]	*[Controlador, Sensor,Atuador]*			☐ OK ☐ Parcial ☐ Pendente
[Outro componente]	*[Controlador, Sensor,Atuador]*			☐ OK ☐ Parcial ☐ Pendente

1.2 Organização do circuito

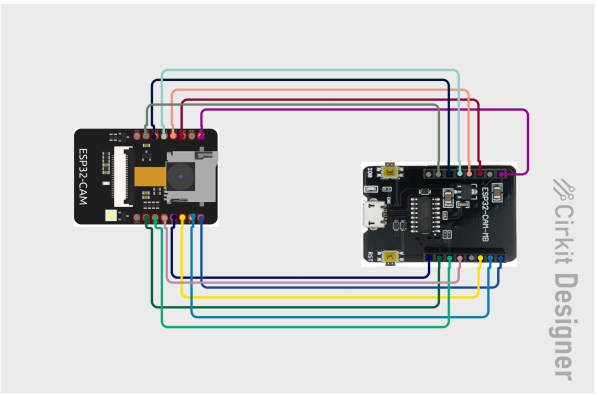


Figura 1 – Esquemático

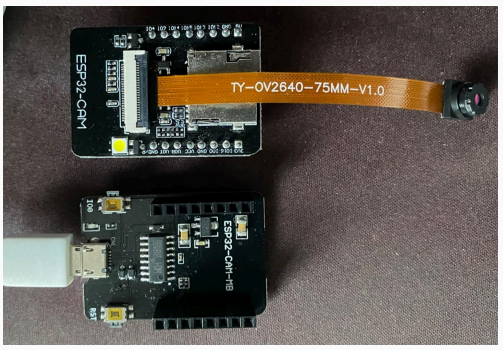


Figura 2 – Montagem física

Arquivo no repositório	/schematics/circuit_image.png

Circuito testado	☐ Cada componente testado individualmente ■ Circuito completo testado
Situação	■ Concluído ☐ Em andamento ☐ Não iniciado
Próximos passos	Construir o suporte que fixa a câmera sobre a palheta e ajustar distância e foco para os ovos ficarem nítidos para o modelo

2. Artefatos de Desenvolvimento

2.1 Códigos-fonte

Artefato	Tecnologias / linguagem	Caminho no repositório	Progresso	Situação
Firmware	C/C++, FreeRTOS, PlatformIO, PubSubClient (MQTT), driver esp_camera (OV2640), esp_http_server	/esp32-esp8266	100 %	■ Concluído ☐ Em andamento ☐ Não iniciado
Back-End	Python: serviço de contagem com Ultralytics YOLO26s + PyTorch + paho-mqtt; API com FastAPI + Uvicorn + WebSocket; SQLite; ReportLab (PDF) + SMTP; MLflow; Docker + Docker Compose	/applications/contador e /python (dashboard/, treino/)	85 %	☐ Concluído ■ Em andamento ☐ Não iniciado
Front-End	JavaScript, React, Vite, Recharts, CSS	/dashboard-web	85%	☐ Concluído ■ Em andamento ☐ Não iniciado

O que já funciona	Câmera (ESP32-CAM + OV2640) - Conecta no Wi-Fi e no broker MQTT. Ao receber um pedido em ovitrampa/cam1/capturar, tira a foto (SVGA 800x600) e publica o JPEG em ovitrampa/cam1/foto. - A captura e o MQTT rodam numa task FreeRTOS própria (mqtt_camera, core 1), com reconexão automática. - Também oferece um servidor HTTP com painel de ajustes, /capture e stream MJPEG. Contagem de ovos (YOLO26s) - Um serviço Python recebe a foto, roda o YOLO26s treinado (classe única "ovo") e publica o resultado em JSON em ovitrampa/cam1/contagem. Também salva a foto original e a anotada. - O status do serviço (online/offline) é publicado num tópico retido, com Last Will. - No conjunto de teste (60 imagens, 2.362 ovos), o modelo obteve mAP50 = 0,94, precisão = 0,92 e recall = 0,91. A contagem errou, em média, 3,5 ovos por imagem (MAE), com limiar de confiança de 0,40, escolhido pelo menor erro de contagem. Dashboard web - Atualização em tempo real por WebSocket. - Captura manual ("Capturar agora") ou automática a cada N minutos, com as etapas da captura (pedido → contando → pronto). - Mostra a foto anotada, os indicadores, os gráficos e o histórico (SQLite), além do status do MQTT, da câmera e do contador. - Modo --simular, para demonstrar sem a câmera. - Relatório em PDF enviado por e-mail (SMTP), só para destinatários autorizados. Infraestrutura - Experimentos e modelo rastreados no MLflow, com o modelo registrado e o alias producao. - O sistema inteiro (Mosquitto, contador e dashboard) sobe com docker compose up
Próximos passos	Testar em campo com ovitrampas e palhetas reais, ajustando foco, iluminação e enquadramento da câmera.

3. Arquitetura do Projeto

3.1 Fluxo de comunicação

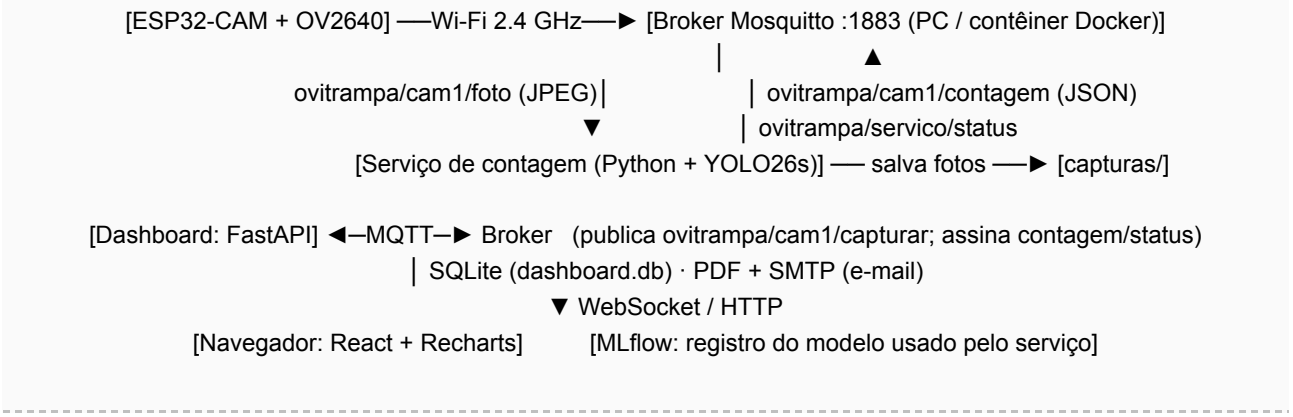


Figura 3 – Arquitetura e fluxo de comunicação

Descrição do fluxo

1. Um usuário clica em “Capturar agora” no dashboard, ou o agendador automático dispara.
2. O back-end FastAPI publica um pedido em ovitrampa/cam1/capturar.
3. A ESP32-CAM, inscrita nesse tópico, captura a foto e publica o JPEG em ovitrampa/cam1/foto.
4. O serviço de contagem recebe a imagem e roda o YOLO26s. Depois salva a original e a anotada e publica o resultado em ovitrampa/cam1/contagem: número de ovos, confiança média, tempo de inferência e arquivos.
5. O back-end do dashboard recebe esse JSON, grava no SQLite e envia ao navegador por WebSocket, que atualiza a foto anotada, os indicadores, os gráficos e o histórico.
6. Sob demanda, o back-end gera um relatório PDF e o envia por e-mail.

Tópico MQTT	Publisher	Subscriber	Payload (formato / exemplo)
ovitrampa/cam1/capturar	Dashboard (FastAPI) / captura_mqtt.py	ESP32-CAM	Qualquer payload (só dispara a captura)
ovitrampa/cam1/foto	ESP32-CAM (ou enviar_imagem.py / modo --simular)	Serviço de contagem; dashboard	JPEG binário, 800x600, QoS 0

Tópico MQTT	Publisher	Subscriber	Payload (formato / exemplo)
ovitrampa/cam1/contagem	Serviço de contagem	Dashboard	JSON: {"ovos": 42, "conf": 0.4, "conf_media": 0.73, "tempo_inferencia_ms": 85.3, "horario": "2026-10-29T14:03:12", "arquivo": "...", "anotada": "...", "largura": 800, "altura": 600}
ovitrampa/servico/status	Serviço de contagem	Dashboard	online / offline (retido)

Situação	■ Concluído ☐ Em andamento ☐ Não iniciado
Próximos passos	Testar o fluxo completo com a câmera em campo por longos períodos, verificando reconexões e quedas de Wi-Fi.

3.2 Infraestrutura (local/nuvem)

Broker MQTT	■ Local (PC / WSL / contêiner / Raspberry Pi) ☐ ESP32 ☐ Nuvem: [(PC / contêiner): Mosquitto no PC ou no contêiner eclipse-mosquitto:2 via Docker Compose, porta 1883]
Back-end e dashboard	■ Local ☐ Nuvem Detalhes: [[FastAPI + serviço YOLO em contêineres Docker no PC, front-end React servido pelo próprio FastAPI na porta 8000, histórico em SQLite, MLflow local (SQLite)]]
Serviços de nuvem	Não se aplica. Tudo roda localmente; o único serviço externo é o SMTP (Gmail) para enviar o relatório por e-mail
Situação	■ Concluído ☐ Em andamento ☐ Não iniciado
Próximos passos	*[O que falta fazer até a entrega final?]*

4. Documentação (Git)

URL do repositório	IgorACNC/camera_contadora
Visibilidade	■ Público ☐ Privado (precisa ser público)
Gestão de tarefas	☐ GitHub Projects ☐ Issues ☐ Não utilizado ■ Notion
Commits do grupo	☐ Todos os integrantes já fizeram commits Total aproximado: [24]

Item da estrutura	Situação	Observações
README.md	■ Completo ☐ Parcial ☐ Ausente	
/docs	■ Completo ☐ Parcial ☐ Ausente	
/applications	■ Completo ☐ Parcial ☐ Ausente	

Item da estrutura	Situação	Observações
/esp32-esp8266	■ Completo ☐ Parcial ☐ Ausente	
/schematics	■ Completo ☐ Parcial ☐ Ausente	

5. Aplicação

5.1 Dashboard

Tecnologia	React + Vite + Recharts no front-end; Python com FastAPI, WebSocket e SQLite no back-end
Dados exibidos	Contagem de ovos de *Aedes aegypti* em cada captura, com a foto original e a anotada pelo YOLO. Também mostra a confiança média das detecções, o limiar usado, o tempo de captura e o tempo de inferência. Tem gráficos de evolução da contagem e o histórico de capturas, com a origem de cada uma (manual, automática ou externa). Mostra ainda o status do MQTT, da câmera e do serviço de contagem (online/offline) e as etapas da captura (pedido → contando → pronto)
Atualização em tempo real	■ Sim ☐ Parcial ☐ Ainda não
Envia comandos aos atuadores	■ Sim ☐ Não ☐ Previsto
Situação	■ Concluído ☐ Em andamento ☐ Não iniciado
Próximos passos	Alerta automático por e-mail quando a contagem passar de um limite e testes do dashboard com fotos reais da câmera

5.2 Protótipo do Front-End

Ferramenta e link	Figma
Telas previstas	Dashboard de contagem de ovos: Objetivo: centralizar o monitoramento da ovitrampa, permitindo ao usuário acompanhar a contagem de ovos, controlar as capturas de imagens e consultar os resultados das análises.

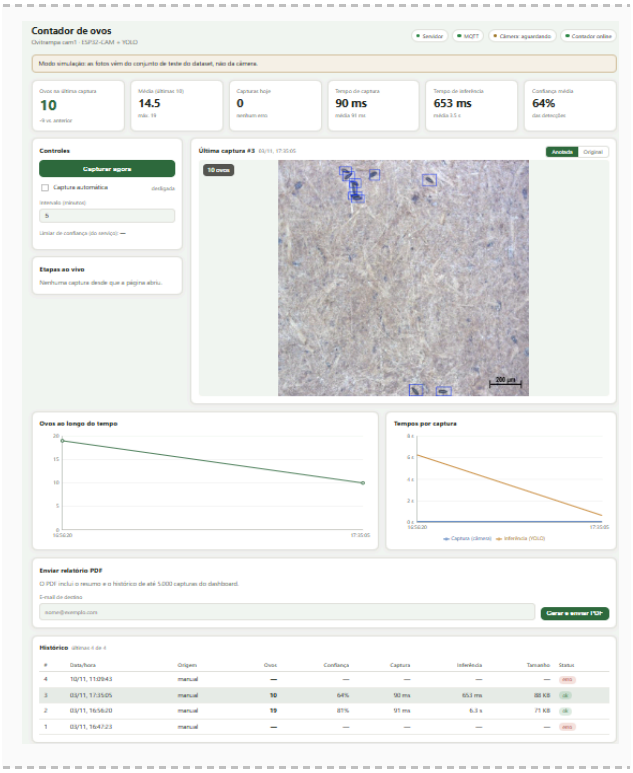


Figura 4 – Dashboard



Figura 5 – Protótipo do Front-End

Painel de Avaliação do Check Point

Uso exclusivo dos docentes – não preencher

Ponto avaliado	Observações dos docentes	Atende?
Prototipagem – uso dos sensores/atuadores		☐ Sim ☐ Parcial ☐ Não
Prototipagem – organização do circuito		☐ Sim ☐ Parcial ☐ Não
Artefatos – códigos-fonte (Front-End, Back-End, Firmware)		☐ Sim ☐ Parcial ☐ Não
Arquitetura – fluxo de comunicação		☐ Sim ☐ Parcial ☐ Não
Arquitetura – infraestrutura (local/nuvem)		☐ Sim ☐ Parcial ☐ Não
Documentação – Git		☐ Sim ☐ Parcial ☐ Não
Aplicação – dashboard		☐ Sim ☐ Parcial ☐ Não
Aplicação – protótipo do Front-End		☐ Sim ☐ Parcial ☐ Não

Evolução do projeto	☐ Adequada ☐ Parcial ☐ Insuficiente
Pontuação do Check Point	Nota: _____ / 15 pts
Parecer e encaminhamentos	
Docente avaliador / data	

